

Dependency Injection (DI)

Dependency Injection is a design pattern where components receive their dependencies from outside rather than creating them internally.

In this project, the “Home” component receives “BookStore” as a prop instead of importing it directly. This makes the component modular, reusable, and easier to test with different stores.

Flux Architecture and Data Flow

Flux follows one-way data flow:

1. **Action** - User triggers an event.
2. **Dispatcher** - Receives the action and sends it to all registered stores.
3. **Store** - Updates its state and notifies subscribed components.
4. **View / Component** – React component listens to store updates and re-renders the UI.

Component lifecycle in Flux context

- **Mounting:** Component subscribes to store updates.
- **Updating:** Store emits a change -> component updates state -> UI re-renders.
- **Unmounting:** Component unsubscribes to prevent memory leaks.

SPA Behavior

The app is a Single Page Application (SPA). Adding a new book updates the Home page dynamically without reloading, demonstrating smooth navigation and real-time updates.